

Intro

Verification process faces significant challenges

Formal verification:

- Comprehensive design exploration
- Encounters state space explosion problem for large scale designs

Simulation:

- High scalability and applicability, can be improved using CRV
- Effectiveness rely on well-designed constraints
- Biased constraints: miss critical scenarios
- Strict constraints: increase debugging complexity

FGTSO:

- Formal Guided Test Sequence Optimization
- Enhance coverage & verification efficiency
- Leverages simulation scalability and formal verification precision to target critical conditions, reducing simulation overhead and improve coverage
- Challenges:
 - Resolve false alarms from formal simulation discrepancies
 - Address BBOX constraints
- Integrate simulation and formal verification
- Introduce abstract models to overcome limitations of BBOX

Related Works

Both these approaches lack methodologies to ensure comprehensive coverage of critical design aspects, as in complex control flows, where formal guidance is essential.

RISC V Torture:

- Generate random instruction sequences to achieve faster coverage closure
- Lacks structured verification guidance: limited effectiveness in extreme corner cases

RISCV-DV:

- Support CRV (constraint random verification) and ISA features
- Reliance on randomness makes it difficult to cover boundary conditions; unstable tests

Fuzzing-based tools leverage golden-reference models and hybrid approaches.

TheHuzz relies on a large number of random/semi-random inputs to trigger abnormal behaviours. It cannot guarantee coverage of all possible scenarios.

HyPFuzz combines fuzzing with formal verification to accelerate coverage and uncover undetected vulnerabilities and CVEs.

This hybrid approach improves verification efficiency, but does not fully address:

- False alarms due to environmental mismatches
- BBOX issues caused by excessive computational complexity or large storage modules

Methodology

Formal guided simulation-based verification framework that integrates constrained random test generation, UVM based simulation, functional checks and targeted instruction sequence optimization.

- RISC-V-DV (add a short description) generate randomized test cases
- Test cases are simulated within UVM environment and executed on the DUV
- Results are cross verified with an ISS for functional correctness.
- Only the FGTSO part corresponds to formal verification, the remaining is simulation based execution environment.
- Simulator generate coverage reports, analyzed by the FGTSO that selects coverage holes for refinement.
- FGTSO generates targeted instruction sequences to address coverage hole.
- Optimized sequences are iteratively fed back into the verification process, refining the verification dataset.

FGTSO- overview

1. **Coverage Hole selection:** identifying and prioritizing coverage holes with the most significant impact on verification efficiency
2. **Environment Consistency Check:** false alarms often arise due to formal and simulation environments inconsistencies. To make an alignment, it applies “assume” properties in SVA during formal verification and edit existing RISC-V-DV constraints accordingly. This step addresses BBOX issues by constructing an abstraction model to ensure more effectiveness of the generated instruction sequences.
3. **Formal Cover Property Generation:** formal cover properties in SVA are defined to precisely characterize the behavior of selected coverage hole, enabling generation of targeted instruction sequences.
4. **Targeted Instruction Sequence Extraction:** FGTSO generates a cover trace (waveform) which is translated into target instruction sequences, then iteratively fed back into the sim environment.

Formal Guided Test Sequence Optimization - Steps

1. The strategy prioritizes addressing the most impactful coverage holes, by focusing on the ones that affect verification efficiency. This accelerates the verification process. The paper employs the MaxUncovd strategy that prioritizes selecting modules with the lowest coverage, assuming that addressing the most uncovered areas yields the greatest verification impact. MaxUncovd enhances the overall efficiency and ensures comprehensive design verification.
2. Misalignment detection between formal verification env with the simulation env is performed to determine the consistency between the two envs. Formal verification generates instruction sequences targeting specific coverage holes, executed in the simulation. If the results from formal verification and simulation are inconsistent (fail to cover intended coverage holes) it indicates a misalignment. Three main causes
 - a. **Primary inputs** (any signal that originates outside the boundary of the specific hardware module you are testing and flows into it, e.g. reset, clock,

interrupt request, rvalid signal [read valid] for AXI protocol). By writing "assumptions," verification engineers tie these wildcards down to reality so the formal tool only explores states that can actually happen.). PIs are treated as free nets, allowing them to assume arbitrary values without constraints. This can lead to incorrect coverage analysis, as generated cover traces may fail to address coverage holes in simulation. Furthermore, unconstrained PIs can result in misleading coverage estimation, especially when test scenarios correspond to unreachable states by design (es AXI protocol). Formal verification may fail to generate meaningful instruction patterns, leaving coverage holes unresolved. Formal assumptions must be introduced to align formal verification with realistic system behavior.

- b. **BBOX:** in EDA tools modules with high storage requirements or complex computational logic are treated as BBOX. They cannot correctly drive the outputs, they may become free nets that can lead to false alarms. This prevents the module logic from driving its outputs. The paper introduced abstraction models that preserve the critical behavior of BBOX modules while ensuring the interaction with the rest of the system is verifiable.
 - i. **High storage requirements module:** Tracking all possible tags incurs excessive storage overhead. Cache tags are crucial for determining hit/miss behavior. A functional abstraction model is introduced, and simplifies the verification while preserving key behaviors. The abstract model is constructed by using an SVA array that tracks cache index accesses, marking indices as valid upon first access to ensure efficient logging. A single tag is enforced for all accesses, reducing overhead while preserving cache functionality.
 - ii. **Complex computational logic module:** are also often treated as BBOX in commercial verification tools due to overloading verification complexity. It introduced a symbolic computation approach using SVA to approximate expected outputs and enhance verification accuracy. SVA captures input signals, performs a simplified symbolic multiplication and uses the result as output for consistency between symbolic and actual computations.
 - c. **Insufficient RISC-V-DV Constraint:** formal verification may produce specialized memory data sequences that existing constraint definitions in RISC-V-DV cannot properly handle. For instance, for a multi level page table (1 GB, "MB and 4 KB page tables), in which we are only able to access 1GB page table, we modify the constraints applying them to the addresses pointed to by the 1 GB and 2 MB page table entries, and set RWX bit to 0, indication that entries should point to the next level of the page table.
3. Use of formal cover properties to obtain cover traces, in order to generate instruction sequences that address target coverage holes, to define behavior of the selected one. This step extracts coverage hole expression and applies it to predefined cover templates. Formal tool executes the cover property, derives the input patterns required to satisfy the condition and generates corresponding cover trace. Then we extract the instruction sequence to cover the coverage hole. Two possible changes:
 - a. **Unreachable Coverage Holes:** first review if constraints in both formal and simulation environments are overly restrictive. If not and the hole is unreachable, it is waived. The information is still provided for debugging.

- b. **Resource Exhaustion:** as state space explosion is problematic, we decompose complex properties into smaller sub-properties, collect their cover traces and integrate them into a comprehensive instruction sequence, to ensure manageable complexity while maintaining verification workability.